ENGINEERING REVIEW

CODE REVIEW REPORT

BACKEND & AUTHORIZATION PASS

A line-by-line review of the Convex backend, shared validation layer, and authorization surface across the service and sales modules.

DOCUMENT ID	SCOPE	DATE	REF
CMA-CRR-001	convex/ + src/lib	07 Aug 2026	commit 0a26911

CRITICAL	HIGH	MEDIUM	LOW
0	1	6	7

01 EXECUTIVE SUMMARY

The codebase is in strong condition. Default-deny authorization, a single typed validation layer, an immutable audit log, and a guarded eight-status job state machine are all implemented correctly. The review found **zero critical** and **one high-severity** item — a production-logging concern — plus six medium and seven low findings, most of which are maintainability wins rather than defects.

VERDICT & SEVERITY DISTRIBUTION

RECOMMENDED: MOVE FORWARD

- Continue MVP on the current architecture
- Keep the defense-in-depth authorization pattern
- Retain the shared Zod + integer-kobo monetary model

GATE BEFORE PUBLIC PRODUCTION

- CR-01 — stop logging password-reset tokens
- CR-02 — cap service-invoice payments at remaining balance
- Review CR-03 (rate limiting) for any publicly reachable surface

02 STRENGTHS

DEFAULT-DENY AUTHORIZATION

convex/lib/auth.ts + src/lib/auth-utils.ts implement a clean matrix: a null role is *never* authorized, admin bypasses, and every mutation routes through requireRole. Tests in tests/convex/authz.test.ts prove anonymous callers are rejected.

S SINGLE SOURCE OF TRUTH

Enums, Zod schemas, and the invoice-totals calculator (computeInvoiceTotals) are shared between client preview and server generation — the same logic that shows the total also bills it.

S GUARDED STATE MACHINE

canTransition is centralized in src/lib/job-utils.ts and re-checked inside every transition mutation, backed by a full unit suite — clients can't skip states.

S IMMUTABLE AUDIT + INTEGER MONEY

All mutations append to auditLogs; stock movements are append-only; all money is integer kobo. These two decisions protect data integrity end to end.

03 FINDINGS

HIGH SEVERITY

CR-01 · PASSWORD-RESET TOKENS LOGGED TO CONSOLE

The reset provider prints the reset code, token, and URL via console.log in convex/auth.ts. In any shared/production environment these land in server logs and could enable account takeover.

Location	`convex/auth.ts:33–38 (sendVerificationRequest)`
Impact	Credential-reset token exposure · High
Fix	Replace with a transactional email provider (SMTP/SendGrid); log only identifier, never token or url.
Effort	~0.5 d

Medium Severity

CR-02 · SERVICE PAYMENTS CAN EXCEED THE INVOICE GRAND TOTAL

payments.record (convex/payments.ts) adds any positive amount to amountPaid without an upper bound, unlike salesOrders.addPayment which caps at balance. Over-recording inflates collections and muddies settlement.

Location	`convex/payments.ts: record`
Impact	Financial-data integrity · Medium
Fix	Reject when amount > grandTotal − amountPaid; also constrain method to an allow-list (cash/transfer/card).

CR-03 · NO REQUEST THROTTLING ON THE PUBLIC SURFACE

Writes (check-in, invoice generation, payments, parts dispatch) and the auth flow have no rate limit. Practically low risk for a single-tenant staff app, but any public-facing route invites scripted abuse and accidental double-submits.

Location	`all mutations`
Impact	Abuse / availability · Medium (mitigated by single-tenant nature)
Fix	Adopt the Throttling Plan for finance/dispatch mutations when/if any surface becomes public.

CR-04 · INVOICES.GENERATE AND REGENERATE ARE NEAR-DUPLICATES

≈70 lines of snapshot/totals logic are repeated between the two handlers, and jobs.syncInvoiceForJob is a third, looser variant typed with any. Drift risk is real.

Location	convex/invoices.ts · convex/jobs.ts
Impact	Maintainability · Medium
Fix	Extract one buildInvoiceLineItems(jobId) + upsertInvoice helper; type syncInvoiceForJob with a MutationCtx instead of any.

CR-05 · INCONSISTENT ERROR PRIMITIVES

parts.adjustStock uses throw new Error while the rest of the backend uses ConvexError (and optional data for structured failure). Callers lose the standard error contract and the client toast sees a bare name.

Location	convex/parts.ts: adjustStock
Impact	Error handling consistency · Medium
Fix	Use ConvexError everywhere; drop the redundant second await requireUser(ctx) (already validated by requireRole).

CR-06 · SEARCH COALESCES WHOLE-COLLECTION READS

parts.search collects all rows then filters in JS; customers.search / leads.search do two separate search-index reads and merge. Fine at current seed scale, but O(n) per keystroke — coordinate with the P1 perf track.

Location	convex/parts.ts · customers.ts · leads.ts
Impact	Read amplification · Medium (at scale)
Fix	Bound with take() at the source, and traces on the client.

Low Severity

ID	OBSERVATION	LOCATION
CR-07	Route guards duplicate inline role arrays across many files	job.$id · customers · finance · sales/*
CR-08	bootstrapFirstAdmin promotes caller to admin (safe now, race-prone	users.ts

ID	OBSERVATION	LOCATION
	later)	
CR-09	audit() silently skips when no actor — missing audit for system ops	lib/audit.ts
CR-10	auditLogs has no index on action/ts for large decks	schema.ts
CR-11	Heterogeneous frontier arrays typed as blanket v.string() in some args	jobs.addJobItem · partsRequests.review
CR-12	window.innerWidth in useState initializer risks hydration mismatch	AppShell.tsx
CR-13	importParts inserts in a loop (no Convex batch)	parts.ts

04 ACTION PLAN

ID	PRIORITY	OWNER	TARGET	EFFORT
CR-01	HIGH	eng	Before public prod	~0.5 d
CR-02	MEDIUM	eng	Next sprint	0.25 d
CR-03	MEDIUM	eng	Per throttling plan	~0.5 d
CR-04	MEDIUM	eng	Next sprint	0.5 d
CR-05	MEDIUM	eng	Next sprint	0.25 d
CR-06	MEDIUM	eng	Perf track	0.5 d
CR-07..13	LOW	eng	Backlog	—

RECOMMENDED CADENCE

Pair the CR-04 refactor with a regression pass on tests/unit/invoice.test.ts and one paid-job E2E, so consolidation lands on verified behavior.

Companion docs: SRS · Throttling · RBAC